



Автоматический режим теперь установлен по умолчанию в Claude Code для планов Pro, Max и Team

Вскоре в планах Pro, Max и Team по умолчанию будет использоваться автоматический режим работы Claude Code, что позволит выполнять автономную работу в течение длительного времени и выявлять больше опасных

💡 Категория
[Код Клода](#)

📦 Продукт
[Код Клода](#)

📅 Дата
7 августа 2026 года

🕒 Время чтения
5 мин

[Поделиться](#)

Мы делаем автоматический режим режимом по умолчанию в Claude Code. Начиная с 14 августа новые сеансы в тарифных планах Pro, Max и Team будут запускаться в автоматическом режиме. Если вы уже установили другой режим по умолчанию, вам может быть предложено один раз подтвердить переход в автоматический режим. Если у вас закрепленный режим по умолчанию, для вас ничего не изменится. Классификатор в автоматическом режиме использует небольшое количество дополнительных токенов для каждого вызова инструмента. С сегодняшнего дня мы больше не взимаем плату за использование классификатора с пользователей Claude Code, выбравших тарифные планы Pro, Max и Team.

Автоматический режим пока доступен только по запросу в Claude Enterprise, Claude API, Claude Platform на AWS, Amazon Bedrock, Google Cloud и Agent Platform, а также в Microsoft Foundry, что дает администраторам время на ознакомление с изменениями. В следующем месяце мы планируем совместно с нашими облачными партнерами сделать этот режим стандартным для всех этих сервисов и больше не взимать плату за классификацию. Тем временем администраторы Enterprise могут сделать автоматический режим Claude Code стандартным с помощью управляемых настроек.

Автоматический режим разработан таким образом, чтобы соблюсти баланс между желанием пользователей не отвлекаться и системой, которая помогает избежать опасных действий: вместо подсказок он направляет каждый вызов инструмента через классификатор,

Блог / Автоматический режим теперь включен по умолчанию в тарифных планах Claude Code для Pro, Max и Team

запрашивает у вас разрешение. Если он не может продвигаться дальше — три блокировки подряд или двадцать за сеанс, — Claude Code возвращается к ручному утверждению.

Последние несколько месяцев мы проверяли, является ли автоматический режим таким же безопасным или даже более безопасным, чем режим, в котором пользователь кликает по подсказкам. Мы провели внутреннее тестирование с привлечением «красной команды», стороннее тестирование с привлечением «красной команды» и внедрение подсказок, а также контролируемое исследование с участием 1053 платных тестировщиков и анализ реальных производственных сессий. По всем показателям, которые мы тестировали, автоматический режим не уступал ручному тестированию или превосходил его.

Автоматический режим также позволяет Claude работать автономно в течение длительного времени. Благодаря этому модели, предназначенные для длительной работы, такие как Claude Opus 5, можно оставлять запущенными на несколько часов для выполнения больших задач. Сокращение трудозатрат пользователей также повышает производительность. Среди пользователей Teams и Enterprise те, кто использует автоматический режим, отправляют на 25% больше PR. Разблокировка Claude позволяет выполнять задачи дольше без перерывов и делать больше работы. Команды Adobe, Nuro, Gusto и Garner Health уже используют автоматический режим по умолчанию.

Below, we share the safety data and customer results motivating the change, and how to set a different default if you prefer.

high suggests many users are clicking through reflexively rather than reviewing each command. These prompts ask developers to make dozens or hundreds of important security decisions every day, often in the middle of projects, which places the review burden on users and increases the chance that something important slips through the cracks. Data also suggests that users more frequently scrutinize and push back on other types of dialogues: for example, when Claude presents a plan for approval, users reject 39% of them. But for individual permissions requests, the rejection rate is only 3%.

The same pattern shows up in settings files. As of June 2026, 49.5% of active CLI users have manually created a Bash allow-rule—5% allow any shell command outright, and another 43% have interpreter rules like `Bash(python:*)` or `Bash(node:*)` that are essentially equivalent in practice—and that share is growing roughly 5 percentage points every 5 weeks. Beyond allow-rules, 62% of users have used `bypassPermissions` or clicked “don’t ask again” on Bash, and 25% of interactive sessions start in bypass permissions mode.

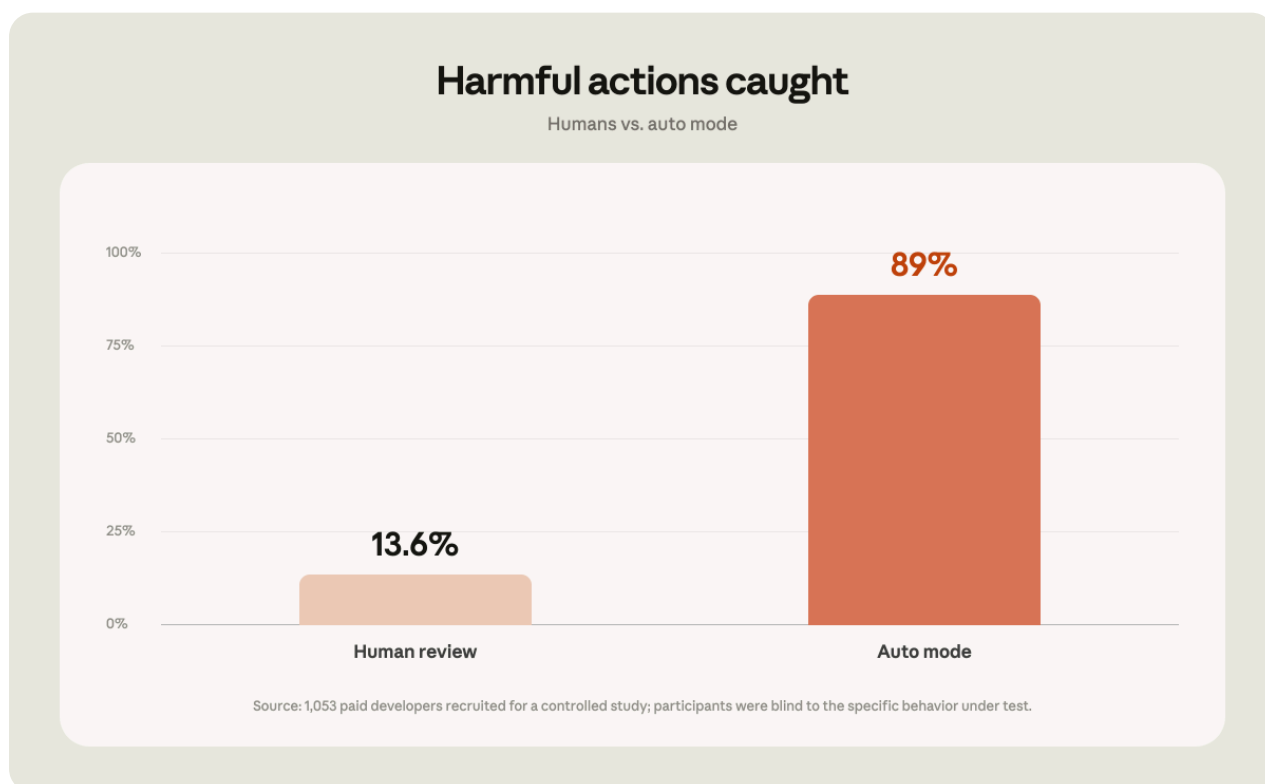
Permission rules still fire before the classifier in auto mode, except for allow rules broad enough to grant arbitrary code execution (e.g. `python:*`). These arbitrary rules are set aside while in auto mode, since they would let commands skip the classifier entirely. Settings files aren’t modified, and the rules apply again the moment you switch to another mode.

Improving safety with auto mode

Research conducted internally and through several third parties suggests that auto mode is safer than how a typical user engages with

Блог / Автоматический режим теперь включен по умолчанию в тарифных планах Claude Code для Pro, Max и Team sessions—auto mode matched or outperformed human approval.

Auto mode outperforms manual permissions on safety



In a controlled experiment with 1,053 paid professional testers, human review caught just 13.6% of dangerous commands, while auto mode caught 89%.

We hired 1,053 paid testers through a research vendor for a coding study. These participants worked in a test environment set up specifically for the study, not in their own codebases or on real projects. Participants knew they were being evaluated, but—following standard blind-study design—weren't told which behavior was being measured. Partway through each session, a single permission prompt was swapped for a clearly dangerous command, and the vendor recorded whether the tester approved it (Note: approving the prompt

(937 of 1,053). Head to head, auto mode blocked 800 commands that a human approved, while humans blocked only 6 that auto mode allowed. As sessions got longer, humans did worse: they blocked about 17% of dangerous commands early in a session, dropping to about 5% after 50 or more prior prompts, while auto mode's block rate stayed flat regardless of session length.

Auto mode prevents more harmful actions

Our research also suggests that auto mode leads to fewer harmful actions than manual permissions mode.

We examined how auto mode performs in real usage. From May to June 2026, we examined sessions our safety pipeline had flagged for review, drawn only from accounts that have opted in to data use for training and analysis. We re-graded each one with a model on two 0–10 scales. The first was harm severity with 10 being company-wide, irreversible harm, such as leaking a company's entire source code, 7 representing production-level severity such as deleting rows from a production database, and 0 a fully reversible action confined to the local workspace, like overwriting a scratch file. The second was how explicitly the user had asked for the action, so we could separate harmful actions the user directly requested from harmful actions the agent took with less explicit direction.

Among the flagged sessions, those that went through manual approval contained serious unintended harm more than twice as often as auto mode sessions. At production-level severity (7+), 6.3% of manually approved sessions contained a harmful action the user

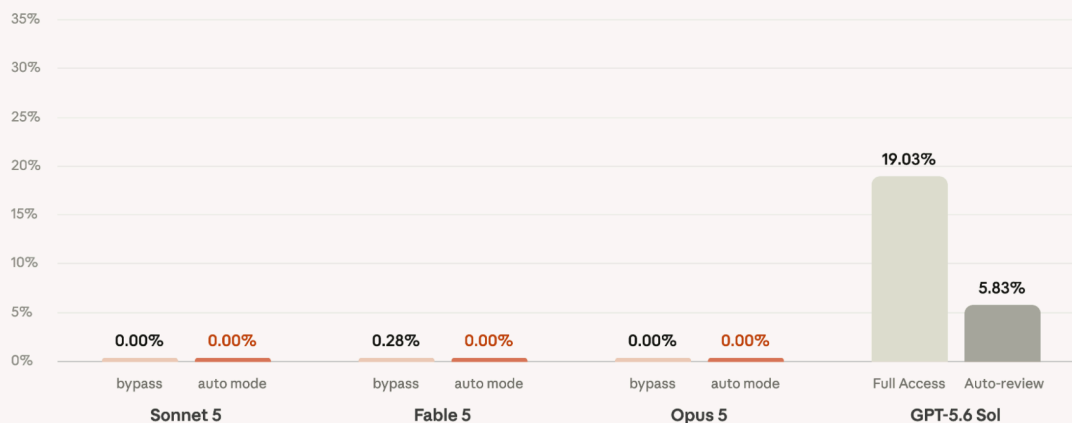
We worked with Apollo Research, a UK-based AI safety startup, on a two-week pilot injecting synthetic attacks into real coding trajectories, across three datasets covering broad failures, data exfiltration, and safety-related threats.

The engagement followed a standard find-fix-verify loop. Apollo shared a set of successful attacks along with recommendations to strengthen auto mode. We hardened the classifier against them primarily by giving it more context about the environment it's protecting (repository visibility, git state, and the data-handling rules described below).

After hardening, Apollo re-tested on a separate held-out attack set that we never had access to. Auto mode caught about half of the attacks it previously missed, at a cost of missing about 2% of attacks it previously caught. Since we couldn't tune against the held-out set, this suggests the hardening generalizes to new attacks rather than patching the specific ones Apollo shared. Across both the shared and held-out sets, the classifier's miss rate fell from 12% to 7%.

These attacks are synthetic and adversarial by design, built to find where the classifier fails, and more malicious than anything Claude would do unprompted. The 7% miss rate is measured against this set and should not be taken as the miss rate on real traffic.

Auto mode reduces prompt injection risk



Source: 72 browser-use and tool-calling tasks × 10 attempts (n = 720 per bar).

With auto mode enabled, no attack in an independent evaluation was successful against any of our models. GPT-5.6 Sol running in Codex v0.144.5 Auto-review permission mode had a 5.83% attack success rate. GPT-5.6 Sol was run on “max” reasoning effort to test performance with its highest level of reasoning, and all Claude models were run on high effort.

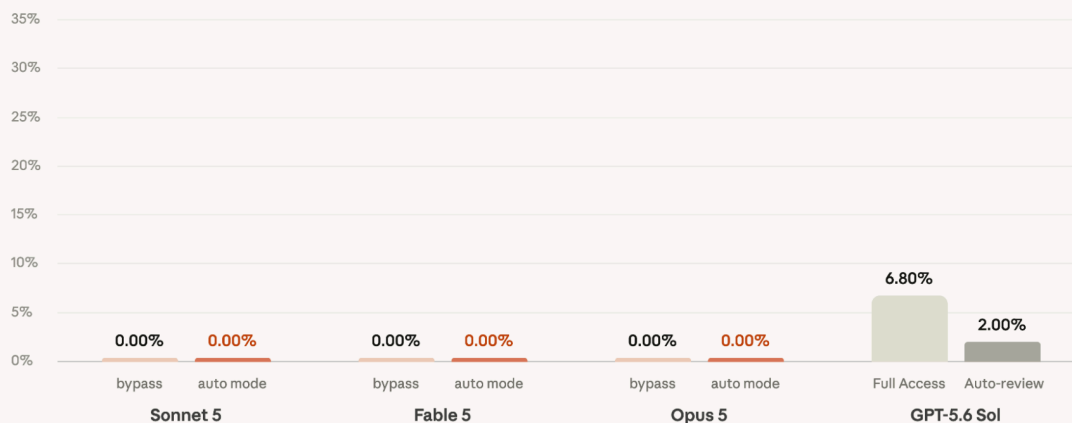
The auto mode classifier adds an additional layer of defense against prompt injection attacks that try to force the agent to deviate from the user’s instructions. When running with auto mode, tool results are scanned by probes for potentially malicious instructions, and auto mode checks that actions are aligned with user intent. We tested auto mode compared to an unguarded model to illustrate the power of this defense-in-depth approach.

We commissioned an evaluation from a third party, Trajectory Labs, who tested different models within the latest publicly available versions of Claude Code and Codex as of July 17th 2026.¹ They tested 72 indirect prompt injection scenarios held out from Anthropic. Each scenario was tested 10 times. Successful attacks would, in a non-testing environment, result in irreversible actions that could harm the user such as sensitive-data exfiltration, financial fraud, or credential phishing. The attacks were optimized against Claude Opus 4.7, which is not in the reported set, so every model in the evaluation faced

browser integration produced by Trajectory Labs, PBC. This integration provides a set of MCP tools which are a thin wrapper around common Chrome APIs such as ``navigate``. Safeguards built into first-party browser integrations such as the Chrome extensions provided by OpenAI and Anthropic were not tested. As such, these results should be viewed as a measurement of the underlying model, rather than the complete set of safeguards which might exist in a given deployment.

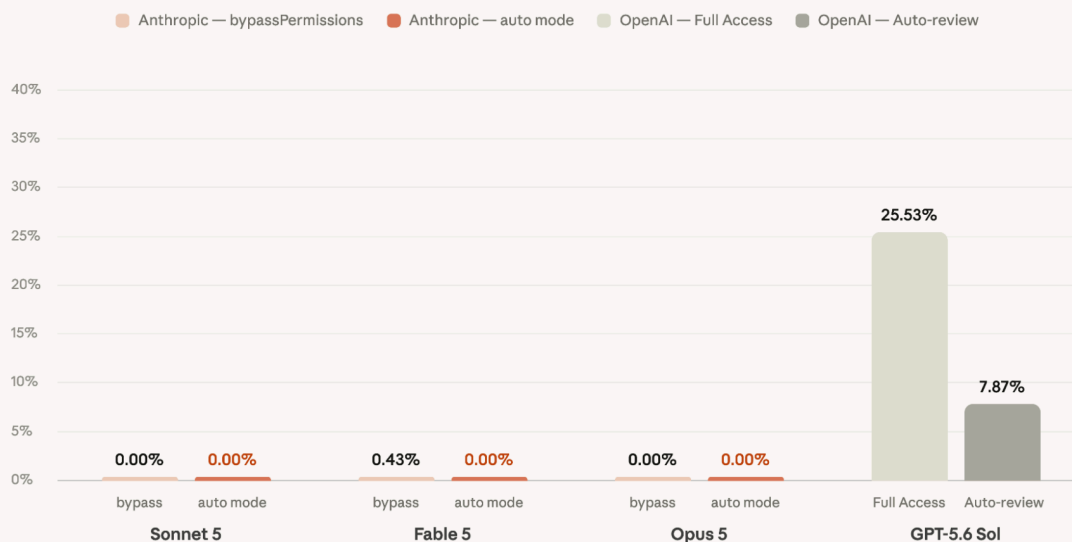
In this evaluation, none of the 720 attack attempts succeeded against Claude Fable 5, Opus 5, or Sonnet 5 running auto mode. On the other hand, 5.83% of the attacks succeeded against GPT-5.6 Sol running Codex's *Auto-review* mode. Notably, this is greater than the 0.09% average attack success rate against our latest models running in *bypassPermissions* mode without additional safeguards. The tests showed a 19.03% attack success rate against GPT-5.6 Sol when running in *Full Access* mode. Attack success rates differ between tasks where tools return results as text and tasks that require interacting with a browser through screenshots, as shown below.

Блог / Автоматический режим теперь включен по умолчанию в тарифных планах Claude Code для Pro, Max и Team



Prompt injection attack success rate: browser-use tasks

By model and permission mode · lower is better



Auto mode protects against prompt injections both in tool calling and browser GUI use. OpenAI models showed higher attack success rates on browser tasks in a third-party evaluation.

Three incidents auto mode prevented inside Anthropic

code-sharing site. Auto mode denied the fallback and prompted the user for next steps.

- **A destructive mass operation:** In a long-running session, Claude attempted a mass process kill across roughly two thousand pods and queued deletions that would have torn down hundreds of GPUs of in-flight training jobs. Auto mode flagged the destructive commands before execution.
- **Over-broad privilege escalation:** Asked to provision read-only billing access, Claude reached for a cloud role that grants full administrative control at the account root. Auto mode caught the mismatch between what was requested and what roles would have been granted.

In each case, Claude either found a safer path on its own or checked in with the user before proceeding.

Making auto mode even safer

We're continuously investing in new auto mode features that make it safer and easier to ship production code. Recent examples include:

- **Hard denies:** Data exfiltration, like sending your code or secrets somewhere external, sits in a category the classifier is designed to never approve. To run an action like that, you have to switch out of auto mode or run the command yourself. Hard deny rules are customizable via settings so you can add more rules that you never want allowed even when requested by users in your organization.
- **Rules for data access and sharing:** The classifier now carries explicit rules distinguishing secrets and potentially sensitive/confidential information—and where each can be

that belongs in your team's private repository shouldn't end up in a public one, and the classifier is now designed to flag when this might happen.

- **Checking git status before destructive git actions:** Before a command that could discard uncommitted work, like `git reset --hard`, the classifier sees the repository's current git status, letting auto mode know what is being reset.
- **Prompt injection screening:** When Claude pulls content from external sources, like web pages, file contents, or tool outputs, an API-side probe checks that content for attempts to hijack Claude's behavior. When something looks like an injection attempt, a warning is added to Claude's context before the result is shared with the user.

Auto mode in production

Teams are already running auto mode as their production default:

- **Adobe's** merchandising platform team is responsible for keeping pricing and promotional pages accurate and current across 90+ countries and 30+ languages on Adobe.com. They built an agentic loop to build and verify those pages, running it in auto mode so engineers receive finished PRs for review.
- **Nuro** runs auto mode across its research and engineering orgs, using it to power overnight research agents that hill-climb evaluation metrics and return finished PRs for review by morning.
- **Gusto** adopted auto mode to end the permission fatigue that was pushing engineers toward bypassing permissions checks entirely.

software development lifecycle (SDLC) that no longer depends on hand-curated command allowlists.

Adobe

“At Adobe, we want to move fast without compromising the quality of the customer experience we deliver on Adobe.com. With Claude Code auto mode, we built an agentic loop that rapidly accelerated our work. Claude builds the user interface and then loops back to verify that it matches the intended design, automatically fixing any issues before we ever look at it. This shortened our development cycle while delivering pixel-perfect results.”

Tomislav Reil, Director of Engineering



Learn how these customers are running auto mode in production.

Getting started

For Pro, Max, and Team users: if you haven't set a default permission mode, you'll receive an in-product notice and new sessions will start in auto mode automatically. If you've set a different default, you may see a one-time prompt asking if you'd like to switch your default to auto mode. If your Team admin has set a default in managed settings, nothing changes for you.

To switch modes, press Shift+Tab in the CLI or use the mode dropdown on the desktop app. Admins can pin an org-wide default with ``defaultMode`` in [managed settings](#), or turn auto mode off entirely with ``disableAutoMode``.

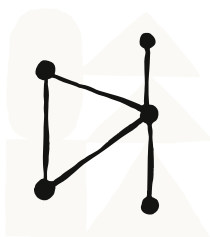
Finally, while we believe auto mode reduces risk for most users, it relies on classification systems and therefore does not eliminate risk. For high-stakes changes to production infrastructure, we still recommend reviewing Claude's actions yourself. [See the auto mode docs](#) for full configuration instructions.

This article was written by Conner Phillippi, with contributions by Nicholas Carlini, Isaac Fung, John Hughes, Alex Isken, Shawn Moore, Javier Rando, and Molly Vorwerck. The authors would also like to thank Yacine Azmi, Chandler Bair, Kefan Chen, Boris Cherny, Ian Gurne, Jiyi Han, Alex Keiman, Lauren Polansky, Deon Poncini, Robert Schonberger, Marie Vachovsky, Qing Wang, Cat Wu, Daniel Xu, and Alice Zhao.

Related posts

Explore more product news and best practices for teams building with Claude.

¹ We evaluated Claude Code v2.1.205 and Codex v0.144.5. OpenAI released a new version of Auto-review last week that could change the results.





Jul 24, 2026

**Claude models explained:
choosing the best model for
your use case**

 Enterprise AI



Jul 22, 2026

**Building verification loops in
Claude Code with skills**

Claude Code

Transform how your organization operates with Claude

[See pricing](#)

[Contact sales](#)

monthly to your inbox.



Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time.

[Write](#)[Learn](#)[Code](#)

Products

[Claude](#)
[Claude Code](#)
[Claude Code for Enterprise](#)
[Claude Cowork](#)
[@Claude](#)
[Claude Design](#)
[Claude Science](#)
[Claude Security](#)
[Download app](#)
[Pricing](#)

Solutions

[AI agents](#)
[Code modernization](#)
[Coding](#)
[Customer support](#)
[Cybersecurity](#)
[Enterprise](#)
[Financial services](#)
[Government](#)
[Healthcare](#)
[Higher education](#)
[K-12 teachers](#)

Resources

[Blog](#)
[Claude partner network](#)
[Community](#)
[Connectors](#)
[Courses](#)
[Customer stories](#)
[Engineering at Anthropic](#)
[Events](#)
[Plugins](#)
[Powered by Claude](#)

Programs

[Startups](#)
[Research Labs](#)

Help and security

[Availability](#)
[Status](#)
[Support center](#)

Terms and policies

[Privacy choices](#)

Блог / Автоматический режим теперь включен по умолчанию в тарифных планах Claude Code для Pro, Max и Team

MICROSOFT 365

Claude Platform

Anthropic

Consumer

Skills

Overview

Careers

Terms of Service: US
K-12

Developer docs

Policy

Models

Pricing

Economic Futures

Data Processing
Agreement: US K-12

Mythos

Ecosystem

Research

Usage policy

Fable

Marketplace

News

Opus

Claude on AWS

Policy on the AI

Sonnet

Google Cloud

Exponential

Haiku

Microsoft Foundry

Responsible Scaling

Regional compliance

Policy

Console login

Security and
compliance

Transparency

ANTHROPIC

© 2026 ANTHROPIC PBC

